

## Getting Started with Chronos-2

[[Open In SageMaker Studio Lab](https://studiolab.sagemaker.aws/studiolab.svg)](https://studiolab.sagemaker.aws/studiolab.svg)  
 (https://studiolab.sagemaker.aws/import/github/amazon-science/chronos-forecasting/blob/main/notebooks/chronos-2-quickstart.ipynb)  
 [[Open In Colab](https://colab.research.google.com/assets/colab-badge.svg)](https://colab.research.google.com/assets/colab-badge.svg)  
 (https://colab.research.google.com/github/amazon-science/chronos-forecasting/blob/main/notebooks/chronos-2-quickstart.ipynb)

**Chronos-2** is a foundation model for time series forecasting that builds on [Chronos] (https://arxiv.org/abs/2403.07815) and [Chronos-Bolt] (https://aws.amazon.com/blogs/machine-learning/fast-and-accurate-zero-shot-forecasting-with-chronos-bolt-and-autogluon/). It offers significant improvements in capabilities and can handle diverse forecasting scenarios not supported by earlier models.

| Capability   Chronos   Chronos-Bolt   Chronos-2                                                                                                                                                                                                                                                                |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| ----- ----- ----- -----                                                                                                                                                                                                                                                                                        |
| Univariate Forecasting                                |
| Cross-learning across items                           |
| Multivariate Forecasting                              |
| Past-only (real/categorical) covariates               |
| Known future (real/categorical) covariates          |
| Fine-tuning support                                   |
| Max. Context Length   512   2048   8192                                                                                                                                                                                                                                                                        |

 Chronos/Chronos-Bolt do not natively support future covariates, but they can be combined with external covariate regressors (see [AutoGluon tutorial] (https://autogluon.ai/stable/tutorials/timeseries/forecasting-chronos.html#incorporating-the-covariates)). This only models per-timestep effects, not effects across time. In contrast, Chronos-2 supports all covariate types natively.

More details about Chronos-2 are available in the [technical report] (https://www.arxiv.org/abs/2510.15821).

```
In [ ]: %pip install 'chronos-forecasting>=2.2[extras]' 'matplotlib'
```

```
In [1]: import os

# Use only 1 GPU if available
os.environ["CUDA_VISIBLE_DEVICES"] = "0"

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from chronos import BaseChronosPipeline, Chronos2Pipeline

# Load the Chronos-2 pipeline
# GPU recommended for faster inference, but CPU is also supported using
device_map="cpu"
pipeline: Chronos2Pipeline =
BaseChronosPipeline.from_pretrained("amazon/chronos-2",
device_map="cuda")
```

## Univariate Forecasting

---

We start with a simple univariate forecasting example using the pandas API.

```
In [2]: # Load data as a long-format pandas data frame
context_df =
pd.read_csv("https://autogluon.s3.amazonaws.com/datasets/timeseries/m4_h
ourly/train.csv")
print("Input dataframe shape:", context_df.shape)
display(context_df.head())
```

Input dataframe shape: (353500, 3)

|   | item_id | timestamp           | target |
|---|---------|---------------------|--------|
| 0 | H1      | 1750-01-01 00:00:00 | 605.0  |
| 1 | H1      | 1750-01-01 01:00:00 | 586.0  |
| 2 | H1      | 1750-01-01 02:00:00 | 586.0  |
| 3 | H1      | 1750-01-01 03:00:00 | 559.0  |
| 4 | H1      | 1750-01-01 04:00:00 | 511.0  |

```
In [3]: pred_df = pipeline.predict_df(context_df, prediction_length=24,
    quantile_levels=[0.1, 0.5, 0.9])

print("Output dataframe shape:", pred_df.shape)
display(pred_df.head())
```

Output dataframe shape: (9936, 7)

|   | item_id | timestamp              | target_name | predictions | 0.1        |     |
|---|---------|------------------------|-------------|-------------|------------|-----|
| 0 | H1      | 1750-01-30<br>04:00:00 | target      | 624.867981  | 611.385132 | 624 |
| 1 | H1      | 1750-01-30<br>05:00:00 | target      | 563.703125  | 546.655090 | 563 |
| 2 | H1      | 1750-01-30<br>06:00:00 | target      | 521.589905  | 505.747498 | 521 |
| 3 | H1      | 1750-01-30<br>07:00:00 | target      | 489.910767  | 473.671814 | 489 |
| 4 | H1      | 1750-01-30<br>08:00:00 | target      | 471.144470  | 452.199402 | 471 |

**predict\_df** supports the following arguments:

- `df` : Long-format DataFrame with id, timestamp, and target column(s)
- `future_df` : Optional DataFrame with future covariates (columns present in both `df` and `future_df` are treated as known future covariates)
- `id_column` : Column with time series identifiers (default: "item\_id")
- `timestamp_column` : Column with timestamps (default: "timestamp")
- `target` : Target column name(s) to forecast (default: "target")
- `prediction_length` : Number of steps to forecast
- `quantile_levels` : Quantiles to compute (default: [0.1, 0.2, ..., 0.9])

Returns a DataFrame with forecasts including point predictions and quantiles.

## Forecasting with Covariates

Chronos-2 can leverage covariates to improve forecast accuracy. We demonstrate this with two real-world examples.

## Energy Price Forecasting

Forecast hourly energy prices for the next day using historical prices and day-ahead forecasts of load (Ampirion Load Forecast) and renewable energy generation (PV+Wind Forecast).

```

In [4]: # Energy price forecasting configuration
target = "target" # Column name containing the values to forecast
           (energy prices)
prediction_length = 24 # Number of hours to forecast ahead
id_column = "id" # Column identifying different time series
           (countries/regions)
timestamp_column = "timestamp" # Column containing datetime information
timeseries_id = "DE" # Specific time series to visualize (Germany)

# Load historical energy prices and past values of covariates
energy_context_df = pd.read_parquet(

"https://autogluon.s3.amazonaws.com/datasets/timeseries/electricity_price/train.parquet"
)
energy_context_df[timestamp_column] =
pd.to_datetime(energy_context_df[timestamp_column])
display(energy_context_df.head())

# Load future values of covariates
energy_test_df = pd.read_parquet(

"https://autogluon.s3.amazonaws.com/datasets/timeseries/electricity_price/test.parquet"
)
energy_test_df[timestamp_column] =
pd.to_datetime(energy_test_df[timestamp_column])
energy_future_df = energy_test_df.drop(columns=target)
display(energy_future_df.head())

```

|   | id | timestamp           | target    | Ampirion Load Forecast | PV+Wind Forecast |
|---|----|---------------------|-----------|------------------------|------------------|
| 0 | DE | 2012-01-09 00:00:00 | 34.970001 | 16382.00               | 3569.527588      |
| 1 | DE | 2012-01-09 01:00:00 | 33.430000 | 15410.50               | 3315.274902      |
| 2 | DE | 2012-01-09 02:00:00 | 32.740002 | 15595.00               | 3107.307617      |
| 3 | DE | 2012-01-09 03:00:00 | 32.459999 | 16521.00               | 2944.620117      |
| 4 | DE | 2012-01-09 04:00:00 | 32.500000 | 17700.75               | 2897.149902      |

|   | id | timestamp           | Ampirion Load Forecast | PV+Wind Forecast |
|---|----|---------------------|------------------------|------------------|
| 0 | DE | 2017-12-12 00:00:00 | 20483.00               | 22284.005859     |
| 1 | DE | 2017-12-12 01:00:00 | 19849.75               | 22878.673828     |

|   | id | timestamp              | Ampirion Load<br>Forecast | PV+Wind<br>Forecast |
|---|----|------------------------|---------------------------|---------------------|
| 2 | DE | 2017-12-12<br>02:00:00 | 19638.25                  | 23632.283203        |
| 3 | DE | 2017-12-12<br>03:00:00 | 19895.25                  | 24635.945312        |
| 4 | DE | 2017-12-12<br>04:00:00 | 20338.00                  | 25584.935547        |

```
In [5]: # Generate predictions with covariates
energy_pred_df = pipeline.predict_df(
    energy_context_df,
    future_df=energy_future_df,
    prediction_length=prediction_length,
    quantile_levels=[0.1, 0.5, 0.9],
    id_column=id_column,
    timestamp_column=timestamp_column,
    target=target,
)
display(energy_pred_df.head())
```

|   | id | timestamp              | target_name | predictions | 0.1       | 0.5       |
|---|----|------------------------|-------------|-------------|-----------|-----------|
| 0 | DE | 2017-12-12<br>00:00:00 | target      | 22.242920   | 18.673723 | 22.242920 |
| 1 | DE | 2017-12-12<br>01:00:00 | target      | 19.525620   | 14.904278 | 19.525620 |
| 2 | DE | 2017-12-12<br>02:00:00 | target      | 17.415363   | 12.209047 | 17.415363 |
| 3 | DE | 2017-12-12<br>03:00:00 | target      | 16.979263   | 11.165121 | 16.979263 |
| 4 | DE | 2017-12-12<br>04:00:00 | target      | 18.058651   | 12.096766 | 18.058651 |

```

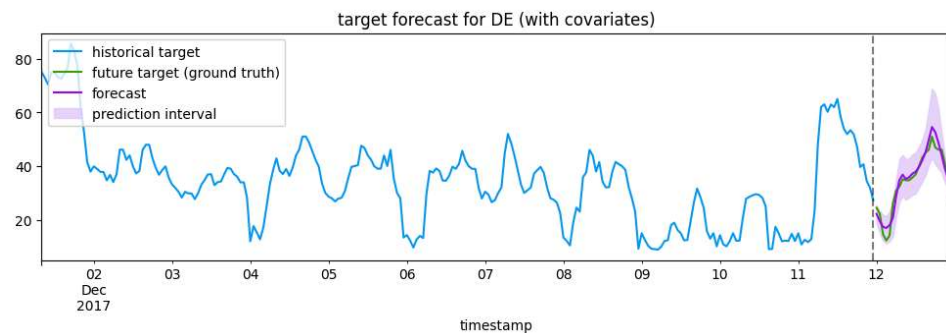
In [6]: # Visualization helper function
def plot_forecast(
    context_df: pd.DataFrame,
    pred_df: pd.DataFrame,
    test_df: pd.DataFrame,
    target_column: str,
    timeseries_id: str,
    id_column: str = "id",
    timestamp_column: str = "timestamp",
    history_length: int = 256,
    title_suffix: str = "",
):
    ts_context = context_df.query(f"{id_column} == @timeseries_id").set_index(timestamp_column)[target_column]
    ts_pred = pred_df.query(f"{id_column} == @timeseries_id and target_name == @target_column").set_index(
        timestamp_column
    )[["0.1", "predictions", "0.9"]]
    ts_ground_truth = test_df.query(f"{id_column} == @timeseries_id").set_index(timestamp_column)[target_column]

    last_date = ts_context.index.max()
    start_idx = max(0, len(ts_context) - history_length)
    plot_cutoff = ts_context.index[start_idx]
    ts_context = ts_context[ts_context.index >= plot_cutoff]
    ts_pred = ts_pred[ts_pred.index >= plot_cutoff]
    ts_ground_truth = ts_ground_truth[ts_ground_truth.index >= plot_cutoff]

    fig = plt.figure(figsize=(12, 3))
    ax = fig.gca()
    ts_context.plot(ax=ax, label=f"historical {target_column}",
color="xkcd:azure")
    ts_ground_truth.plot(ax=ax, label=f"future {target_column} (ground truth)", color="xkcd:grass green")
    ts_pred["predictions"].plot(ax=ax, label="forecast",
color="xkcd:violet")
    ax.fill_between(
        ts_pred.index,
        ts_pred["0.1"],
        ts_pred["0.9"],
        alpha=0.7,
        label="prediction interval",
        color="xkcd:light lavender",
    )
    ax.axvline(x=last_date, color="black", linestyle="--", alpha=0.5)
    ax.legend(loc="upper left")
    ax.set_title(f"{target_column} forecast for {timeseries_id} {title_suffix}")
    fig.show()

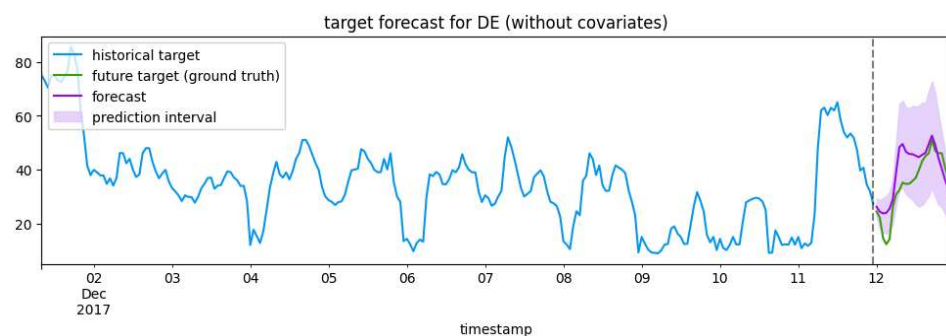
```

```
In [7]: # Visualize forecast with covariates
plot_forecast(
    energy_context_df,
    energy_pred_df,
    energy_test_df,
    target_column=target,
    timeseries_id=timeseries_id,
    title_suffix="(with covariates)",
)
```



```
In [8]: # Compare: forecast without covariates
energy_pred_no_cov_df = pipeline.predict_df(
    energy_context_df[[id_column, timestamp_column, target]],
    future_df=None,
    prediction_length=prediction_length,
    quantile_levels=[0.1, 0.5, 0.9],
    id_column=id_column,
    timestamp_column=timestamp_column,
    target=target,
)

plot_forecast(
    energy_context_df,
    energy_pred_no_cov_df,
    energy_test_df,
    target_column=target,
    timeseries_id=timeseries_id,
    title_suffix="(without covariates)",
)
```



The comparison shows that Chronos-2 makes reasonable but imprecise predictions in univariate mode. However, with covariates, Chronos-2 effectively uses the load and renewable generation forecasts, producing significantly more accurate predictions.



## Retail Demand Forecasting

Forecast next quarter's weekly store sales using historical sales, historical customer footfall (Customers), and known covariates indicating store operation (Open), promotion periods (Promo), and holidays (SchoolHoliday, StateHoliday).

In [9]:

```

# Retail forecasting configuration
target = "Sales" # Column name containing sales values to forecast
prediction_length = 13 # Number of days to forecast ahead
id_column = "id" # Column identifying different products/stores
timestamp_column = "timestamp" # Column containing datetime information
timeseries_id = "1" # Specific time series to visualize (product/store ID)

# Load historical sales and past values of covariates
sales_context_df =
pd.read_parquet("https://autogluon.s3.amazonaws.com/datasets/timeseries/
retail_sales/train.parquet")
sales_context_df[timestamp_column] =
pd.to_datetime(sales_context_df[timestamp_column])
display(sales_context_df.head())

# Load future values of covariates
sales_test_df =
pd.read_parquet("https://autogluon.s3.amazonaws.com/datasets/timeseries/
retail_sales/test.parquet")
sales_test_df[timestamp_column] =
pd.to_datetime(sales_test_df[timestamp_column])
sales_future_df = sales_test_df.drop(columns=target)
display(sales_future_df.head())

```

|   | id | timestamp  | Sales   | Open     | Promo    | SchoolHoliday |   |
|---|----|------------|---------|----------|----------|---------------|---|
| 0 | 1  | 2013-01-13 | 32952.0 | 0.857143 | 0.714286 | 5.0           | ( |
| 1 | 1  | 2013-01-20 | 25978.0 | 0.857143 | 0.000000 | 0.0           | ( |
| 2 | 1  | 2013-01-27 | 33071.0 | 0.857143 | 0.714286 | 0.0           | ( |
| 3 | 1  | 2013-02-03 | 28693.0 | 0.857143 | 0.000000 | 0.0           | ( |
| 4 | 1  | 2013-02-10 | 35771.0 | 0.857143 | 0.714286 | 0.0           | ( |

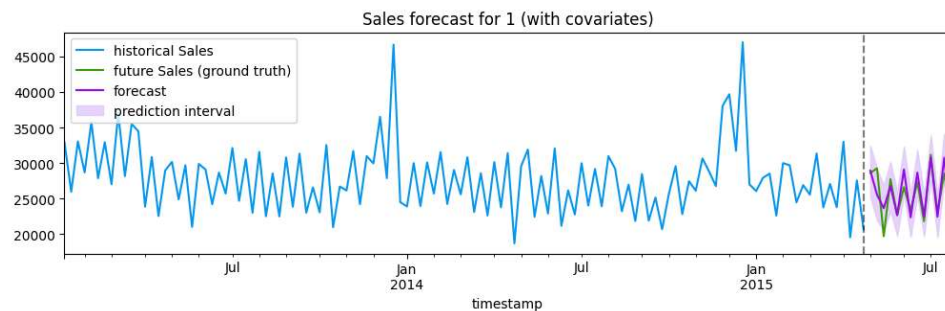
|   | id | timestamp  | Open     | Promo    | SchoolHoliday | StateHoliday |
|---|----|------------|----------|----------|---------------|--------------|
| 0 | 1  | 2015-05-03 | 0.714286 | 0.714286 | 0.0           | 1.0          |
| 1 | 1  | 2015-05-10 | 0.857143 | 0.714286 | 0.0           | 0.0          |
| 2 | 1  | 2015-05-17 | 0.714286 | 0.000000 | 0.0           | 1.0          |
| 3 | 1  | 2015-05-24 | 0.857143 | 0.714286 | 0.0           | 0.0          |

|   | id | timestamp  | Open     | Promo    | SchoolHoliday | StateHoliday |
|---|----|------------|----------|----------|---------------|--------------|
| 4 | 1  | 2015-05-31 | 0.714286 | 0.000000 | 0.0           | 1.0          |

```
In [10]: # Generate predictions with covariates
sales_pred_df = pipeline.predict_df(
    sales_context_df,
    future_df=sales_future_df,
    prediction_length=prediction_length,
    quantile_levels=[0.1, 0.5, 0.9],
    id_column=id_column,
    timestamp_column=timestamp_column,
    target=target,
)
display(sales_pred_df.head())
```

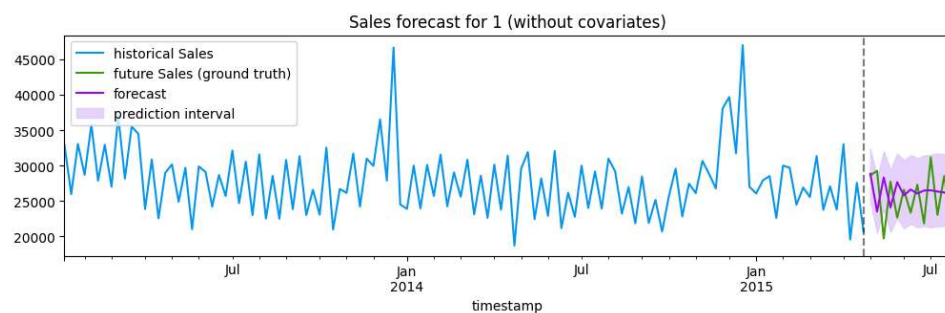
|   | id | timestamp  | target_name | predictions  | 0.1          |      |
|---|----|------------|-------------|--------------|--------------|------|
| 0 | 1  | 2015-05-03 | Sales       | 28939.392578 | 25214.277344 | 2893 |
| 1 | 1  | 2015-05-10 | Sales       | 25541.919922 | 21921.324219 | 2554 |
| 2 | 1  | 2015-05-17 | Sales       | 23640.238281 | 20500.339844 | 2364 |
| 3 | 1  | 2015-05-24 | Sales       | 26778.261719 | 23318.355469 | 2677 |
| 4 | 1  | 2015-05-31 | Sales       | 22679.361328 | 19722.287109 | 2267 |

```
In [11]: # Visualize forecast with covariates
plot_forecast(
    sales_context_df,
    sales_pred_df,
    sales_test_df,
    target_column=target,
    timeseries_id=timeseries_id,
    title_suffix="(with covariates)",
)
```



```
In [12]: # Compare: forecast without covariates
sales_pred_no_cov_df = pipeline.predict_df(
    sales_context_df[[id_column, timestamp_column, target]],
    future_df=None,
    prediction_length=prediction_length,
    quantile_levels=[0.1, 0.5, 0.9],
    id_column=id_column,
    timestamp_column=timestamp_column,
    target=target,
)

plot_forecast(
    sales_context_df,
    sales_pred_no_cov_df,
    sales_test_df,
    target_column=target,
    timeseries_id=timeseries_id,
    title_suffix="(without covariates)",
)
```



Chronos-2's univariate forecast is nearly flat with high uncertainty. In contrast, the forecast with covariates leverages promotion and holiday information to capture the true sales dynamics over the forecast horizon.

## Cross-Learning with Joint Prediction

---

Chronos-2 supports **cross-learning** through the `cross_learning=True` parameter, which enables the model to share information across all time series in a batch during prediction. This can be particularly beneficial when forecasting multiple related time series with short historical context.

```
In [13]: # Example: Enable cross-learning for joint prediction
# This assigns the same group ID to all time series, allowing
# information sharing
joint_pred_df = pipeline.predict_df(
    context_df,
    prediction_length=24,
    quantile_levels=[0.1, 0.5, 0.9],
    cross_learning=True, # Enable cross-learning
    batch_size=100,
)
```

### Important Considerations for Cross-Learning

When using `cross_learning=True`, keep these caveats in mind:

- **Task-dependent results:** Cross-learning may not always improve forecasts and could worsen performance for some tasks. Evaluate this feature for your specific use case.
- **Batch size dependency:** Results become dependent on batch size. Very large batch sizes may not provide benefits as they deviate from the maximum group size used during pretraining. For optimal results, consider using a batch size around 100 (as used in the paper).
- **Input homogeneity:** This feature works best with homogeneous inputs (e.g., multiple univariate time series of the same type). Mixing different task types may lead to unexpected behavior.
- **Short context benefit:** Cross-learning is most helpful when individual time series have limited historical context, as the model can leverage patterns from related series in the batch.

## (Advanced) Numpy/torch API

---

For advanced use cases, Chronos-2 provides a lower-level numpy/torch API via the `predict_quantiles` method.

The `predict_quantiles` method accepts:

- `inputs` : Time series to forecast (see formats below)
- `prediction_length` : Number of timesteps to forecast
- `quantile_levels` : List of quantiles to compute

Two input formats are supported:

1. **3D array**: (`batch_size`, `num_variates`, `history_length`) for forecasting without covariates

2. **List of dicts**: Each dict contains:

- `target` : 1D or 2D array of shape (`history_length`,) or (`num_variates`, `history_length`)
- `past_covariates` (optional): Dict mapping covariate names to 1D arrays of length `history_length`
- `future_covariates` (optional): Dict mapping covariate names to 1D arrays of length `prediction_length`

```
In [14]: # Univariate forecasting
inputs = np.random.randn(32, 1, 100)
quantiles, mean = pipeline.predict_quantiles(
    inputs, prediction_length=24, quantile_levels=[0.1, 0.5, 0.9]
)
print("Univariate output shapes:", quantiles[0].shape, mean[0].shape)
```

```
Univariate output shapes: torch.Size([1, 24, 3]) torch.Size([1,
24])
```

```
In [15]: # Multivariate forecasting
inputs = np.random.randn(32, 3, 512)
quantiles, mean = pipeline.predict_quantiles(
    inputs, prediction_length=48, quantile_levels=[0.1, 0.5, 0.9]
)
print("Multivariate output shapes:", quantiles[0].shape, mean[0].shape)
```

```
Multivariate output shapes: torch.Size([3, 48, 3]) torch.Size([3,
48])
```

```
In [16]: # Univariate forecasting with covariates
prediction_length = 64
inputs = [
    {
        "target": np.random.randn(200),
        "past_covariates": {"temperature": np.random.randn(200),
        "precipitation": np.random.randn(200)},
        "future_covariates": {"temperature":
np.random.randn(prediction_length)},
    }
    for _ in range(16)
]
quantiles, mean = pipeline.predict_quantiles(
    inputs, prediction_length=prediction_length, quantile_levels=[0.1,
0.5, 0.9]
)
print("Univariate with covariates output shapes:", quantiles[0].shape,
mean[0].shape)
```

```
Univariate with covariates output shapes: torch.Size([1, 64, 3])
torch.Size([1, 64])
```

```
In [17]: # Multivariate forecasting with categorical covariates
prediction_length = 96
inputs = [
    {
        "target": np.random.randn(2, 1000),
        "past_covariates": {
            "temperature": np.random.randn(1000),
            "weather_type": np.random.choice(["sunny", "cloudy",
"rainy"], size=1000),
        },
        "future_covariates": {
            "temperature": np.random.randn(prediction_length),
            "weather_type": np.random.choice(["sunny", "cloudy",
"rainy"], size=prediction_length),
        },
    }
    for _ in range(10)
]
quantiles, mean = pipeline.predict_quantiles(
    inputs, prediction_length=prediction_length, quantile_levels=[0.1,
0.5, 0.9]
)
print("Multivariate with categorical covariates output shapes:",
quantiles[0].shape, mean[0].shape)
```

```
Multivariate with categorical covariates output shapes:
torch.Size([2, 96, 3]) torch.Size([2, 96])
```

## Fine-Tuning

---

Chronos-2 supports fine-tuning on your own data. You may either fine-tune all weights of the model (`_full` fine-tuning) or a [low rank adapter (LoRA)] ([https://huggingface.co/docs/peft/en/package\\_reference/lora](https://huggingface.co/docs/peft/en/package_reference/lora)), which significantly reduces the number of trainable parameters.

**Note:** Fine-tuning functionality is intended for advanced users. The default fine-tuning hyperparameters may not always improve accuracy for your specific use case. We recommend experimenting with different hyperparameters. In case of limited data (too few and/or too short series), fine-tuning may not improve over zero-shot (and may even worsen accuracy sometimes).

### Fine-Tuning API

The `fit` method accepts:

- `inputs` : Time series for fine-tuning (same format as `predict_quantiles`)
- `finetune_mode` : `"full"` or `"lora"`
- `lora_config` : The [ `LoraConfig` ] ([https://huggingface.co/docs/peft/en/package\\_reference/lora#peft.LoraConfig](https://huggingface.co/docs/peft/en/package_reference/lora#peft.LoraConfig)), in case `finetune_mode="lora"`
- `prediction_length` : Forecast horizon for fine-tuning
- `validation_inputs` : Optional validation data (same format as `inputs`)
- `learning_rate` : Optimizer learning rate (default: `1e-6`, we recommend a higher learning rate such as `1e-5` for LoRA)
- `num_steps` : Number of training steps (default: 1000)
- `batch_size` : Batch size for training (default: 256)

Returns a new pipeline with the fine-tuned model.

Please read the docstring for details about specific arguments.



```
In [18]: # Prepare data for fine-tuning using the retail sales dataset
known_covariates = ["Open", "Promo", "SchoolHoliday", "StateHoliday"]
past_covariates = ["Customers"]

train_inputs = []
for item_id, group in sales_context_df.groupby("id"):
    train_inputs.append({
        "target": group[target].values,
        "past_covariates": {col: group[col].values for col in
past_covariates + known_covariates},
        # Future values of covariates are not used during training.
        # However, we need to include their names to indicate that these
columns will be available at prediction time
        "future_covariates": {col: None for col in known_covariates},
    })
```

```
In [19]: # Fine-tune the model by default full fine-tuning will be performed
finetuned_pipeline = pipeline.fit(
    inputs=train_inputs,
    prediction_length=13,
    num_steps=1000,
    learning_rate=1e-5,
    batch_size=32,
    logging_steps=100,
)
```

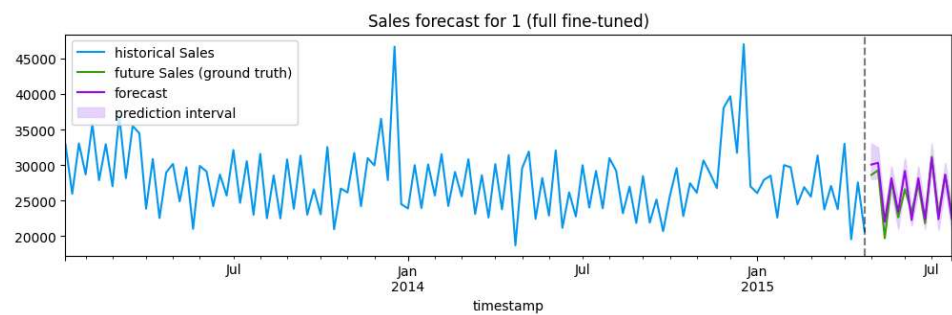
Could not estimate the number of tokens of the input, floating-point operations will not be computed

 [1000/1000 01:13, Epoch 1/9223372036854775807]

| Step | Training Loss |
|------|---------------|
| 100  | 0.645200      |
| 200  | 0.531200      |
| 300  | 0.506000      |
| 400  | 0.463900      |
| 500  | 0.458000      |
| 600  | 0.523900      |
| 700  | 0.460600      |
| 800  | 0.431000      |
| 900  | 0.443300      |
| 1000 | 0.501900      |

```
In [20]: # Use the fine-tuned model for predictions
finetuned_pred_df = finetuned_pipeline.predict_df(
    sales_context_df,
    future_df=sales_future_df,
    prediction_length=13,
    quantile_levels=[0.1, 0.5, 0.9],
    id_column="id",
    timestamp_column="timestamp",
    target="Sales",
)

plot_forecast(
    sales_context_df,
    finetuned_pred_df,
    sales_test_df,
    target_column="Sales",
    timeseries_id="1",
    title_suffix="(full fine-tuned)",
)
```



```
In [21]: # Fine-tune the model with LoRA
lora_finetuned_pipeline = pipeline.fit(
    inputs=train_inputs,
    prediction_length=13,
    num_steps=1000,
    learning_rate=1e-4,
    batch_size=32,
    logging_steps=100,
    finetune_mode="lora",
)
```

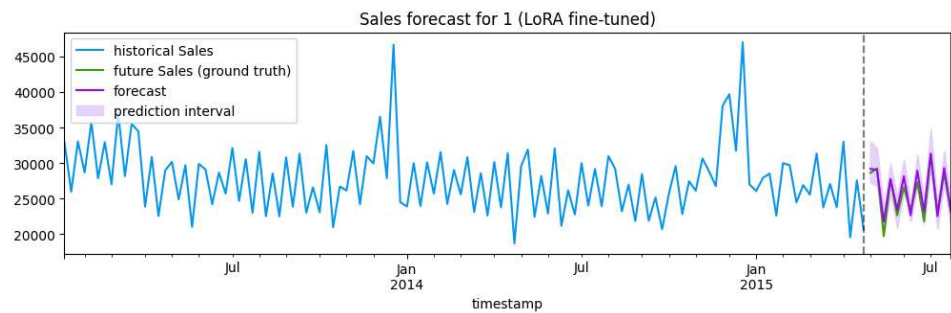
Could not estimate the number of tokens of the input, floating-point operations will not be computed

 [1000/1000 01:53, Epoch 1/9223372036854775807]

| Step | Training Loss |
|------|---------------|
| 100  | 0.752000      |
| 200  | 0.617800      |
| 300  | 0.596500      |
| 400  | 0.539700      |
| 500  | 0.544400      |
| 600  | 0.605700      |
| 700  | 0.546800      |
| 800  | 0.511900      |
| 900  | 0.537200      |
| 1000 | 0.583500      |

```
In [22]: # Use the LoRA fine-tuned model for predictions
lora_finetuned_pred_df = lora_finetuned_pipeline.predict_df(
    sales_context_df,
    future_df=sales_future_df,
    prediction_length=13,
    quantile_levels=[0.1, 0.5, 0.9],
    id_column="id",
    timestamp_column="timestamp",
    target="Sales",
)

plot_forecast(
    sales_context_df,
    lora_finetuned_pred_df,
    sales_test_df,
    target_column="Sales",
    timeseries_id="1",
    title_suffix="(LoRA fine-tuned)",
)
```



```
In [ ]:
```